



Sistema de Gestión de Préstamos Bibliotecarios con Autenticación Multifactor
(MFA)

Documentación Técnica del Proyecto

Presentado por

Wilson Armando Rojas Vera
Jeyson Javier Varela Suarez

1. Introducción y Visión General

BiblioAuth es una aplicación web desarrollada en Flask (Python) que gestiona el préstamo de libros de una biblioteca universitaria. Su característica distintiva es el uso de autenticación multifactor (MFA) diferenciada según el tipo de usuario:

- Estudiantes: inician sesión con código estudiantil + contraseña, y como segundo factor reciben un código OTP de 6 dígitos por correo electrónico institucional.
- Bibliotecarios (administradores): inician sesión con usuario + contraseña, y como segundo factor usan TOTP (Time-based One-Time Password) mediante una app como Google Authenticator.

El sistema permite a un bibliotecario buscar estudiantes, registrar préstamos de libros, registrar devoluciones físicas, y a los estudiantes confirmar la devolución desde su propio portal autenticado (principio de no repudio: la devolución queda sellada con la sesión MFA del estudiante).

1.1 Stack tecnológico

Componente	Tecnología
Backend	Flask 3.0.3, Flask-SQLAlchemy, Flask-Login, Flask-WTF
Base de datos	SQLite (vía SQLAlchemy ORM)
Seguridad / MFA	bcrypt (hash de contraseñas y OTP), pyotp (TOTP), qrcode (generación de QR), PyJWT
Correo	smtplib + email.mime (envío de OTP por SMTP/TLS)
Frontend	Jinja2 (plantillas server-side), HTML/CSS embebido en base.html
Otros	python-dotenv (variables de entorno), Pillow/pypng (soporte de imágenes para QR)

1.2 Estructura general de carpetas

```
MFA_Biblio/
├── app/
│   ├── __init__.py          # Application factory (create_app)
│   ├── models.py           # Modelos SQLAlchemy (BD)
│   ├── modules/
│   │   ├── __init__.py
│   │   ├── audit.py        # Tabla y función de auditoría
│   │   ├── jwt_manager.py
│   │   ├── otp_email.py    # Lógica de OTP por correo (estudiantes)
│   │   └── totp.py         # Lógica de TOTP/QR (bibliotecarios)
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── admin.py        # Rutas del bibliotecario
│   │   ├── auth.py         # Rutas de autenticación (login/MFA)
│   │   └── prestamos.py    # Rutas del estudiante
│   └── templates/
│       ├── base.html
│       ├── auth/
│       ├── admin/
│       └── prestamos/
└── scripts/
    └── seed_db.py          # Script para poblar la BD con datos de prueba
```

```
|— config.py          # Configuración de la app (Flask, BD, correo, OTP)
|— run.py            # Punto de entrada (arranque del servidor)
|— requirements.txt   # Dependencias del proyecto
|— .gitignore
```

1.3 Flujo de autenticación

Estudiante:

- 1) Ingresa código estudiantil + contraseña en /auth/portal
- 2) Si son correctos, se genera y envía un OTP de 6 dígitos al correo institucional
- 3) El estudiante lo ingresa en /auth/estudiante/verificar-otp
- 4) Si es válido, se crea la sesión con Flask-Login (rol = estudiante)

Bibliotecario:

- 1) Ingresa usuario + contraseña en /auth/portal
- 2) Si es su primer ingreso, se le muestra un código QR para configurar Google Authenticator (/auth/bibliotecario/setup-totp)
- 3) En cada ingreso posterior, debe escribir el código TOTP vigente en /auth/bibliotecario/verificar-totp
- 4) Si es válido, se crea la sesión con Flask-Login (rol = bibliotecario)

2. Archivos en la raíz del proyecto

2.1 run.py

Punto de entrada de la aplicación. Es el archivo que se ejecuta para levantar el servidor de desarrollo.

```
from app import create_app

app = create_app()

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
```

Qué hace:

Importa la función fábrica `create_app()` desde el paquete `app/` y construye la instancia de Flask. Si el script se ejecuta directamente (no se importa como módulo), arranca el servidor en modo debug, escuchando en todas las interfaces de red (0.0.0.0) en el puerto 5000. Comentario en el propio código advierte que `debug=True` nunca debe usarse en producción (expone un debugger interactivo y recarga automática).

2.2 config.py

Centraliza toda la configuración de la aplicación en una clase `Config`, leyendo valores desde variables de entorno (`.env`) con valores por defecto seguros para desarrollo.

```
class Config:
    SECRET_KEY = ...
    SQLALCHEMY_DATABASE_URI = ...
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    MAIL_SERVER / MAIL_PORT / MAIL_USE_TLS / MAIL_USERNAME / MAIL_PASSWORD / MAIL_SENDER
    OTP_EXPIRY_MINUTES = 5
    MAX_OTP_ATTEMPTS = 3
```

Qué hace cada parte:

`load_dotenv()`: carga variables del archivo `.env` al entorno del proceso.

`BASE_DIR`: ruta absoluta de la raíz del proyecto, usada para construir la ruta de la base de datos SQLite y evitar errores por rutas relativas.

`SECRET_KEY`: clave usada por Flask para firmar la sesión y cookies; tiene un valor por defecto inseguro que debe cambiarse en producción.

`SQLALCHEMY_DATABASE_URI`: cadena de conexión a la BD; por defecto SQLite en `instance/biblioauth.db`, pero puede sobreescribirse con la variable `DATABASE_URL`.

Bloque `MAIL_*`: configuración del servidor SMTP (por defecto Gmail, puerto 587 con TLS) usado para enviar los OTP a los estudiantes.

`OTP_EXPIRY_MINUTES`: minutos de validez de un código OTP (5 por defecto).

`MAX_OTP_ATTEMPTS`: número máximo de intentos fallidos antes de invalidar el código OTP (3 por defecto).

2.3 requirements.txt

Lista de dependencias exactas (con versión fijada) necesarias para ejecutar el proyecto. Se instalan con `pip install -r requirements.txt`. Incluye Flask y sus extensiones (Login, SQLAlchemy, WTF), bcrypt y pyotp/qrcode para MFA, PyJWT, y python-dotenv para la configuración.

2.4 .gitignore

Archivo de configuración de Git que define qué archivos/carpetas no deben subirse al repositorio (típicamente entornos virtuales, archivos .env con credenciales, la base de datos local, archivos de caché de Python __pycache__, etc.).

3. Carpeta app/ — Núcleo de la aplicación

3.1 app/__init__.py — Application Factory

Este es el archivo más importante de arranque: implementa el patrón 'Application Factory' de Flask, que permite crear instancias configurables de la app (útil para testing y para evitar imports circulares).

Elemento	Función
<code>db = SQLAlchemy()</code>	Instancia global del ORM, sin asociar todavía a ninguna app Flask (se asocia luego con <code>db.init_app(app)</code>).
<code>login_manager = LoginManager()</code>	Instancia global del gestor de sesiones de Flask-Login.
<code>create_app()</code>	Función fábrica: crea la app Flask, carga la configuración desde <code>config.Config</code> , inicializa las extensiones (<code>db</code> , <code>login_manager</code>), define la vista de login (<code>auth.elegir_portal</code>) y el mensaje flash al exigir login, registra los tres blueprints (<code>auth_bp</code> , <code>prestamos_bp</code> , <code>admin_bp</code>), crea las tablas de la BD si no existen (<code>db.create_all()</code>) dentro del contexto de la app, e importa <code>app.models</code> para que SQLAlchemy reconozca los modelos antes de crear las tablas. Retorna la instancia <code>app</code> .

Nota técnica: `login_manager.login_view` se configura como `"auth.elegir_portal"`, por lo que cualquier vista protegida con `@login_required` redirigirá automáticamente a esa pantalla de login unificado si el usuario no está autenticado.

3.2 app/models.py — Modelos de base de datos

Define el esquema completo de la base de datos usando SQLAlchemy ORM. Contiene 5 modelos/tablas.

Función load_user(user_id)

Decorada con `@login_manager.user_loader`. Flask-Login la invoca automáticamente para reconstruir el objeto de usuario a partir del ID guardado en la cookie de sesión. Como el sistema tiene dos tipos de usuario (Estudiante y Bibliotecario) que comparten el mismo espacio de IDs de sesión, primero intenta cargar el ID como Estudiante; si no existe, lo intenta como Bibliotecario.

Clase Estudiante (tabla estudiantes)

Campo	Descripción
Id	Integer, PK
codigo_estudiantil	String(20), único, obligatorio
Nombre	String(100), obligatorio
correo_institucional	String(120), único, obligatorio — destino de los OTP
password_hash	String(255) — hash bcrypt de la contraseña
Estado	String(10), default 'ACTIVO' — controla si puede iniciar sesión / pedir préstamos
Carrera	String(100), opcional
creado_en	DateTime, default = ahora (UTC)
prestamos / otp_tokens	Relaciones uno-a-muchos hacia Prestamo y OTPToken

Hereda de UserMixin (requerido por Flask-Login para exponer is_authenticated, get_id(), etc.) y db.Model.

Clase Bibliotecario (tabla bibliotecarios)

Campo	Descripción
Id	Integer, PK
usuario	String(50), único, obligatorio
password_hash	String(255)
totp_secret	String(32), opcional — clave secreta base32 de TOTP
totp_activo	Boolean, default False — indica si ya configuró su autenticador
creado_en	DateTime
prestamos	Relación uno-a-muchos hacia Prestamo (préstamos que registró)

Clase Libro (tabla libros)

Campo	Descripción
Id	Integer, PK
titulo / autor	String, obligatorios
Isbn	String(20), único, opcional
disponible	Boolean, default True
categoria	String(80), opcional
total_ejemplares	Integer, default 1
creado_en	DateTime
prestamos	Relación uno-a-muchos hacia Prestamo

Clase Prestamo (tabla prestamos)

Tabla central que vincula Estudiante, Libro y Bibliotecario. El campo estado tiene 4 valores posibles: ACTIVO, PENDIENTE_CONFIRMACION, DEVUELTO, VENCIDO.

Campo	Descripción
estudiante_id / libro_id / bibliotecario_id	FKs hacia las tres tablas relacionadas

Campo	Descripción
Estado	String(30), default 'ACTIVO'
fecha_prestamo	DateTime, default ahora (UTC)
fecha_devolucion_esp	DateTime — fecha límite esperada de devolución
fecha_devolucion_real	DateTime — se llena cuando el estudiante confirma la devolución
confirmado_estudiante	Boolean, default False — sello de no repudio

Clase OTPToken (tabla otp_tokens)

Guarda los códigos OTP enviados a los estudiantes, siempre hashados (nunca en texto plano).

Campo	Descripción
estudiante_id	FK hacia estudiantes
token_hash	String(255) — hash bcrypt del OTP de 6 dígitos
creado_en / expira_en	DateTime — control de vigencia
Usado	Boolean, default False — el OTP es de un solo uso
intentos_fallidos	Integer, default 0 — para limitar fuerza bruta

4. Carpeta app/modules/ — Lógica de negocio (servicios)

Esta carpeta agrupa la lógica reutilizable que no pertenece directamente a una ruta HTTP: generación y verificación de OTP/TOTP, envío de correos, y auditoría.

4.1 app/modules/__init__.py

Este archivo NO es un simple inicializador de paquete; contiene una segunda copia, casi idéntica, de la función `create_app()` (la real está en `app/__init__.py`).

Observaciones importantes:

Define su propia instancia `db = SQLAlchemy()` y `login_manager = LoginManager()`, independientes de las de `app/__init__.py` — esto es una duplicación de estado global que puede generar confusión, ya que cualquier código que importe 'db' desde `app.modules` en vez de desde `app` obtendría una instancia distinta y desconectada.

Su `login_manager.login_view` apunta a "auth.login", una vista que no existe en `app/routes/auth.py` (allí la vista se llama `elegir_portal`), lo cual provocaría un error si esta función llegara a ejecutarse.

Define rutas adicionales (/ , manejadores de error 404 y 500) que no están presentes en la `create_app()` real, y que referencian plantillas `404.html` / `500.html` que no existen en `app/templates/`.

Esta función `create_app()` no es importada ni utilizada en ningún otro punto del proyecto (`run.py` importa `create_app` desde `app`, no desde `app.modules`), por lo que actualmente es código muerto. Se recomienda eliminarlo o, si se quiso usar para los manejadores de error 404/500, migrar esa lógica al `app/__init__.py` real.

4.2 app/modules/audit.py — Auditoría de eventos

Implementa un registro de auditoría inmutable de las acciones críticas del sistema (login, configuración TOTP, préstamos, devoluciones, etc.).

Clase LogAuditoria (tabla log_auditoria)

Campo	Descripción
timestamp	DateTime, default ahora (UTC)
actor_id / actor_rol	Quién ejecutó la acción (id) y su rol ('bibliotecario' / 'estudiante')
Acción	String(50) — código de la acción (p. ej. LOGIN_EXITOSO)
descripcion	String(255), opcional — detalle libre
ip_origen	String(45), opcional — IP del actor

Función registrar(actor_id, actor_rol, accion, descripcion="", ip="")

Crea una nueva fila en LogAuditoria y la guarda en la BD (db.session.add + commit). Las acciones documentadas en el docstring son: LOGIN_EXITOSO, LOGIN_FALLIDO, TOTP_CONFIGURADO, PRESTAMO_REGISTRADO, DEVOLUCION_REGISTRADA, DEVOLUCION_CONFIRMADA, SESION_CERRADA.

4.3 app/modules/jwt_manager.py

El archivo existe pero no tiene contenido. A pesar de que PyJWT está listado en requirements.txt, no hay actualmente ninguna lógica de tokens JWT implementada en el proyecto. Es probable que este archivo estuviera planeado para una futura funcionalidad (p. ej. tokens de API, recuperación de contraseña, o sesiones sin estado) que aún no se desarrolló.

4.4 app/modules/otp_email.py — OTP por correo (estudiantes)

Implementa todo el ciclo de vida del segundo factor de autenticación de los estudiantes: generación, almacenamiento seguro, verificación y envío por correo electrónico.

Función	Qué hace
generar_otp()	Genera un código numérico de 6 dígitos usando el módulo secrets (CSPRNG, criptográficamente seguro), a diferencia de random que no es apto para uso criptográfico. Fórmula: secrets.randbelow(900000) + 100000, garantizando siempre 6 dígitos (100000–999999).
guardar_otp(estudiante_id, otp_plano)	Invalida (borra) todos los OTP previos no usados del estudiante; calcula la fecha de expiración según OTP_EXPIRY_MINUTES; hashea el OTP con bcrypt antes de guardarlo (nunca se guarda en texto plano); crea y persiste un nuevo registro OTPToken. Retorna el objeto token creado.
verificar_otp(estudiante_id, otp_ingresado)	Busca el token activo (no usado) más reciente del estudiante. Verifica en orden: que exista, que no haya expirado, que no se haya superado MAX_OTP_ATTEMPTS. Compara el código ingresado contra el hash con bcrypt.checkpw. Si es válido, marca el token como usado=True (un solo uso) y retorna {valido: True, mensaje}. Si es inválido, incrementa intentos_fallidos y retorna {valido: False, mensaje} con el detalle del

Función	Qué hace
	motivo de rechazo (no existe, expiró, demasiados intentos, código incorrecto).
enviar_otp_correo(destinatario, nombre, otp_plano)	Construye un correo MIME multipart (texto plano + HTML con estilo de marca, color #003366/#ad3333) y lo envía vía smtpplib.SMTP con STARTTLS al servidor configurado (MAIL_SERVER/MAIL_PORT). Usa las credenciales MAIL_USERNAME/MAIL_PASSWORD para autenticarse. Si ocurre cualquier excepción, la captura, la imprime en consola (no la expone al usuario, por seguridad) y retorna False; si el envío fue exitoso retorna True.
generar_y_enviar_otp(estudiante)	Función orquestadora de alto nivel que encadena las tres anteriores: genera el OTP en texto plano, lo guarda hasheado (guardar_otp), y lo envía al correo institucional del estudiante (enviar_otp_correo, usando solo el primer nombre). Retorna True/False según el resultado del envío. Es la función que se llama directamente desde la ruta de login del estudiante.

4.5 app/modules/totp.py — TOTP / QR (bibliotecarios)

Implementa el segundo factor de autenticación de los bibliotecarios usando el estándar TOTP (RFC 6238), compatible con apps como Google Authenticator, mediante la librería pyotp, junto con la generación del código QR de configuración (librería qrcode).

Función	Qué hace
generar_secret()	Genera una clave secreta aleatoria en base32 (32 caracteres) usando pyotp.random_base32(). Esta clave se guarda en BD (campo totp_secret) y es la base matemática para calcular los códigos TOTP.
generar_uri_totp(secret, usuario)	Construye la URI estándar otpauth://totp/BiblioAuth:usuario?secret=...&issuer=BiblioAuth usando pyotp.TOTP(secret).provisioning_uri(). Esta URI es la que se codifica dentro del QR y que Google Authenticator interpreta al escanear.
generar_qr_base64(secret, usuario)	Genera la imagen QR (librería qrcode) a partir de la URI anterior, la guarda en un buffer en memoria (io.BytesIO, sin tocar disco) en formato PNG, y la codifica en base64 para poder incrustarla directamente en HTML como .
verificar_totp(secret, codigo_ingresado)	Verifica si el código de 6 dígitos ingresado es válido para el secret dado, usando totp.verify(codigo, valid_window=1). El valid_window=1 tolera una diferencia de hasta ± 30 segundos entre el reloj del servidor y el del dispositivo del bibliotecario. Retorna False inmediatamente si falta el secret o el código.
activar_totp_bibliotecario(bibliotecario)	Flujo de alto nivel para la primera configuración: genera el secret, lo guarda en el bibliotecario y hace commit, genera el

Función	Qué hace
	QR en base64, y retorna un diccionario {secret, qr_base64} para mostrarlo en la plantilla setup_totp.html.
confirmar_activacion_totp(bibliotecario, codigo_ingresado)	Se llama tras escanear el QR: verifica el primer código ingresado contra el secret recién generado; si es válido, marca totp_activo=True y hace commit (confirmando que la app quedó bien configurada); retorna True/False.

5. Carpeta app/routes/ — Controladores (Blueprints)

Contiene los tres Blueprints de Flask que agrupan las rutas HTTP de la aplicación: autenticación, panel del bibliotecario, y portal del estudiante.

5.1 app/routes/__init__.py

Archivo vacío. Solo está presente para que Python reconozca app/routes/ como un paquete importable.

5.2 app/routes/auth.py — Blueprint auth_bp (prefijo /auth)

Gestiona todo el flujo de autenticación de dos pasos para ambos tipos de usuario.

Función / Ruta	Qué hace
index() — GET /auth/	Redirige siempre a la pantalla de selección de portal (auth.elegir_portal).
elegir_portal() — GET/POST /auth/portal	Pantalla única de login con selector de rol. En POST recibe identificador, password y rol. Si rol='bibliotecario': busca al Bibliotecario por usuario y valida la contraseña con bcrypt.checkpw; si es correcta, guarda su id en session['bibliotecario_pre_auth'] y redirige a setup_totp (si nunca configuró TOTP) o a verificar_totp_view (si ya lo tiene activo). Si rol='estudiante': busca al Estudiante por codigo_estudiantil, valida contraseña con bcrypt, verifica que esté ACTIVO, guarda session['estudiante_pre_auth'], dispara generar_y_enviar_otp(estudiante) y redirige a verificar_otp_view. En cualquier caso de credenciales inválidas o rol desconocido, muestra un mensaje flash de error y vuelve a renderizar el formulario.
logout() — GET /auth/logout (requiere login)	Limpia toda la sesión (session.clear()), cierra la sesión de Flask-Login (logout_user()) y redirige al portal con un mensaje de confirmación.
login_bibliotecario() — GET/POST /auth/bibliotecario/login	Ruta heredada/alias: simplemente redirige al login unificado (auth.elegir_portal).
setup_totp() — GET/POST /auth/bibliotecario/setup-totp	Solo accesible si existe session['bibliotecario_pre_auth'] (primer paso superado). En GET genera/reutiliza el secret TOTP y muestra el QR (activar_totp_bibliotecario). En POST recibe el

Función / Ruta	Qué hace
	primer código ingresado y lo valida con <code>confirmar_activacion_totp</code> ; si es correcto, redirige a la verificación normal de TOTP; si no, muestra error y vuelve a mostrar el QR.
<code>verificar_totp_view()</code> — GET/POST <code>/auth/bibliotecario/verificar-totp</code>	Segundo factor del bibliotecario. En POST valida el código TOTP con <code>verificar_totp()</code> ; si es válido, limpia <code>bibliotecario_pre_auth</code> , fija <code>session['rol']='bibliotecario'</code> , llama a <code>login_user()</code> (Flask-Login) y redirige al dashboard del admin. Si es inválido, muestra mensaje de error.
<code>login_estudiante()</code> — GET/POST <code>/auth/estudiante/login</code>	Ruta heredada/alias: redirige al login unificado.
<code>verificar_otp_view()</code> — GET/POST <code>/auth/estudiante/verificar-otp</code>	Segundo factor del estudiante. En POST valida el OTP con <code>verificar_otp()</code> ; si es válido, limpia <code>estudiante_pre_auth</code> , fija <code>session['rol']='estudiante'</code> , llama a <code>login_user()</code> y redirige a 'mis préstamos'. Si es inválido, muestra el mensaje específico de rechazo devuelto por <code>verificar_otp</code> (expirado, intentos agotados, incorrecto, etc.).

5.3 `app/routes/admin.py` — Blueprint `admin_bp` (prefijo `/admin`)

Rutas exclusivas del bibliotecario: dashboard, búsqueda de estudiantes, registro de préstamos y devoluciones.

Función / Ruta	Qué hace
<code>solo_bibliotecario(f)</code>	Decorador personalizado (usa <code>functools.wraps</code>) que verifica que <code>session['rol'] == 'bibliotecario'</code> . Si no, muestra un flash de acceso restringido y redirige al portal. Se aplica a todas las rutas de este blueprint junto con <code>@login_required</code> , formando una doble verificación: estar autenticado Y tener el rol correcto.
<code>dashboard()</code> — GET <code>/admin/dashboard</code>	Calcula y muestra estadísticas generales: total de estudiantes activos, total de libros, libros disponibles, préstamos activos y préstamos pendientes de confirmación.
<code>buscar_estudiante()</code> — GET/POST <code>/admin/buscar-estudiante</code>	En POST busca un estudiante por <code>codigo_estudiantil</code> exacto. Si no existe, muestra un flash de advertencia. Renderiza el resultado (o None) en la plantilla.
<code>registrar_prestamo(estudiante_id)</code> — GET/POST <code>/admin/registrar-prestamo/<id></code>	Valida que el estudiante exista (<code>get_or_404</code>), que esté ACTIVO, y que no tenga ya 3 préstamos activos (límite de negocio). En GET muestra los libros disponibles. En POST valida que el libro elegido siga disponible, crea el Préstamo con estado ACTIVO y <code>fecha_devolucion_esp = ahora + 7 días</code> , marca el libro como no disponible, y guarda los cambios. Muestra un flash de confirmación con la fecha esperada de devolución.

Función / Ruta	Qué hace
registrar_devolucion(prestamo_id) — POST /admin/registrar-devolucion/<id>	El bibliotecario marca que recibió físicamente el libro. Solo aplica si el préstamo está en estado ACTIVO; lo cambia a PENDIENTE_CONFIRMACION (queda a la espera de que el propio estudiante confirme la devolución desde su portal, como mecanismo de no repudio).
listar_prestamos() — GET /admin/prestamos	Lista todos los préstamos en estado ACTIVO o PENDIENTE_CONFIRMACION, ordenados por fecha de préstamo descendente.

5.4 app/routes/prestamos.py — Blueprint prestamos_bp (prefijo /prestamos)

Rutas exclusivas del estudiante: ver sus préstamos, su historial, y confirmar devoluciones.

Función / Ruta	Qué hace
solo_estudiante(f)	Decorador análogo a solo_bibliotecario, pero verifica session['rol'] == 'estudiante'. Protege todas las rutas de este blueprint.
mis_prestamos() — GET /prestamos/mis-prestamos	Muestra los préstamos del estudiante autenticado que estén en estado ACTIVO o PENDIENTE_CONFIRMACION, ordenados por fecha descendente. Pasa también ahora (datetime actual) a la plantilla, probablemente para calcular/mostrar si un préstamo está vencido.
historial() — GET /prestamos/historial	Muestra absolutamente todos los préstamos históricos del estudiante autenticado (cualquier estado), ordenados por fecha descendente.
confirmar_devolucion(prestamo_id) — POST /prestamos/confirmar-devolucion/<id>	El estudiante confirma que efectivamente devolvió el libro. Verifica primero que el préstamo le pertenezca (prestamo.estudiante_id == current_user.id) — evita que un estudiante confirme préstamos ajenos — y que el préstamo esté en estado PENDIENTE_CONFIRMACION. Si todo es correcto, cambia el estado a DEVUELTO, marca confirmado_estudiante=True, registra fecha_devolucion_real y vuelve a marcar el libro como disponible. Este es el punto donde se materializa el 'no repudio': la confirmación sólo puede ocurrir dentro de una sesión MFA válida del propio estudiante.

6. Carpeta app/templates/ — Vistas (Jinja2)

Todas las plantillas heredan de base.html mediante `{% extends "base.html" %}` y sobrescriben el bloque `{% block content %}`. base.html define la estructura HTML general, la barra de navegación (que cambia según el rol en sesión) y los estilos CSS embebidos (paleta con tono rojo institucional).

6.1 base.html

Plantilla maestra (451 líneas). Define el `<head>`, los estilos CSS globales, la barra de navegación superior (con enlaces distintos para bibliotecario: Dashboard / Nuevo préstamo / Préstamos; y para estudiante: Mis préstamos / Historial), el área de mensajes flash, el bloque `{% block content %}` donde se inyecta cada vista hija, y el enlace de Cerrar sesión.

6.2 Carpeta templates/auth/

Archivo	Contenido / Propósito
portal.html	Pantalla de inicio unificada. Formulario POST hacia <code>auth.elegir_portal</code> con un selector/toggle de rol (bibliotecario o estudiante), campo identificador y password.
login_bibliotecario.html	Formulario de login específico para bibliotecario (vista de respaldo; la ruta activa actualmente redirige a <code>portal.html</code>).
login_estudiante.html	Formulario de login específico para estudiante (vista de respaldo; análoga a la anterior).
setup_totp.html	Muestra el código QR (imagen base64) y el secret en texto, junto con un formulario para ingresar el primer código TOTP y confirmar la activación.
verificar_totp.html	Formulario simple para ingresar el código TOTP de 6 dígitos en cada inicio de sesión del bibliotecario.
verificar_otp.html	Formulario para ingresar el código OTP de 6 dígitos recibido por correo electrónico (estudiante).

6.3 Carpeta templates/admin/

Archivo	Contenido / Propósito
dashboard.html	Panel principal del bibliotecario: tarjetas con las estadísticas calculadas en <code>admin.dashboard()</code> y accesos directos a 'Nuevo préstamo' y 'Préstamos'.
buscar_estudiante.html	Formulario para buscar un estudiante por código; si se encuentra, muestra sus datos y un enlace para iniciar el registro de préstamo (<code>admin.registrar_prestamo</code>).
registrar_prestamo.html	Formulario para seleccionar un libro disponible y asociarlo al estudiante encontrado, completando el registro del préstamo.
listar_prestamos.html	Tabla de préstamos activos / pendientes de confirmación, con un formulario por fila para disparar <code>admin.registrar_devolucion</code> .

6.4 Carpeta templates/prestamos/

Archivo	Contenido / Propósito
mis_prestamos.html	Tabla con los préstamos activos/pendientes del estudiante en sesión, con un formulario por fila para disparar <code>prestamos.confirmar_devolucion</code> , y un enlace al historial completo.
historial.html	Tabla con el historial completo de préstamos del estudiante (todos los estados).

7. Carpeta scripts/

7.1 scripts/seed_db.py

Script independiente (no forma parte del servidor web) que se ejecuta manualmente con python `scripts/seed_db.py` para poblar la base de datos con datos de prueba.

Qué hace:

Ajusta `sys.path` para poder importar el paquete `app/` aunque el script se ejecute desde la carpeta `scripts/`.

Define `hashear(password)`: helper que aplica `bcrypt.hashpw + gensalt()` a una contraseña en texto plano.

Función `seed()`: dentro del contexto de la app (`app.app_context()`), primero borra todos los registros existentes de Estudiante, Bibliotecario y Libro (no toca Préstamo ni OTPToken ni LogAuditoria).

Inserta 4 estudiantes de ejemplo (con distintos estados ACTIVO/INACTIVO y carreras), un bibliotecario de prueba (`admin_biblioteca / admin123`, con `totp_activo=False`, por lo que en su primer login se le pedirá configurar el QR), y 5 libros de ejemplo (uno de ellos ya marcado como no disponible).

Si ocurre cualquier error durante el proceso, hace `rollback()` de la transacción y vuelve a lanzar la excepción (`raise`) tras imprimir el error.

Al ejecutarse como script principal (`if __name__ == '__main__':`), llama a `seed()`.